ANDROID IPC

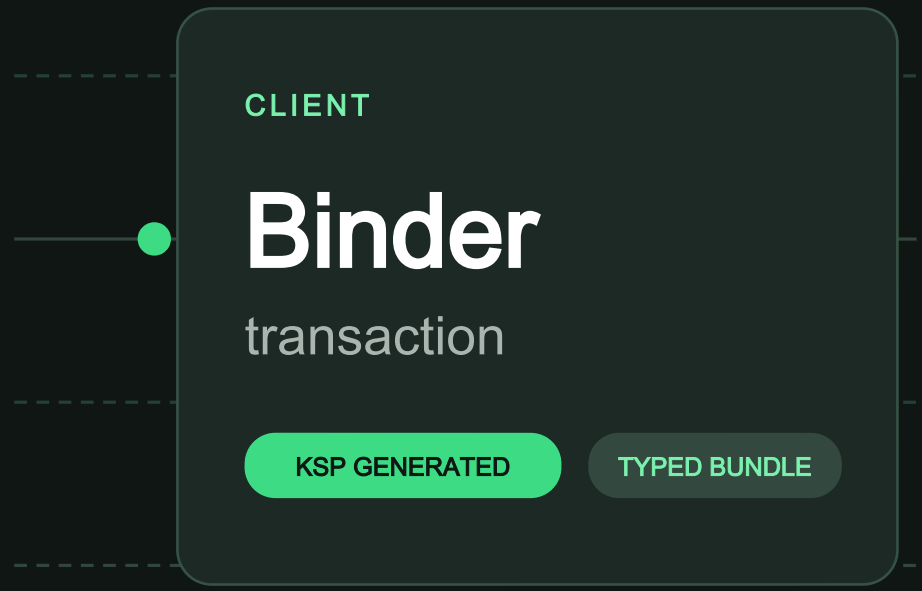
# Android IPC 全景与 Falcon 实践

从 Binder 成本模型到编译期 IPC 框架

1 选对语义

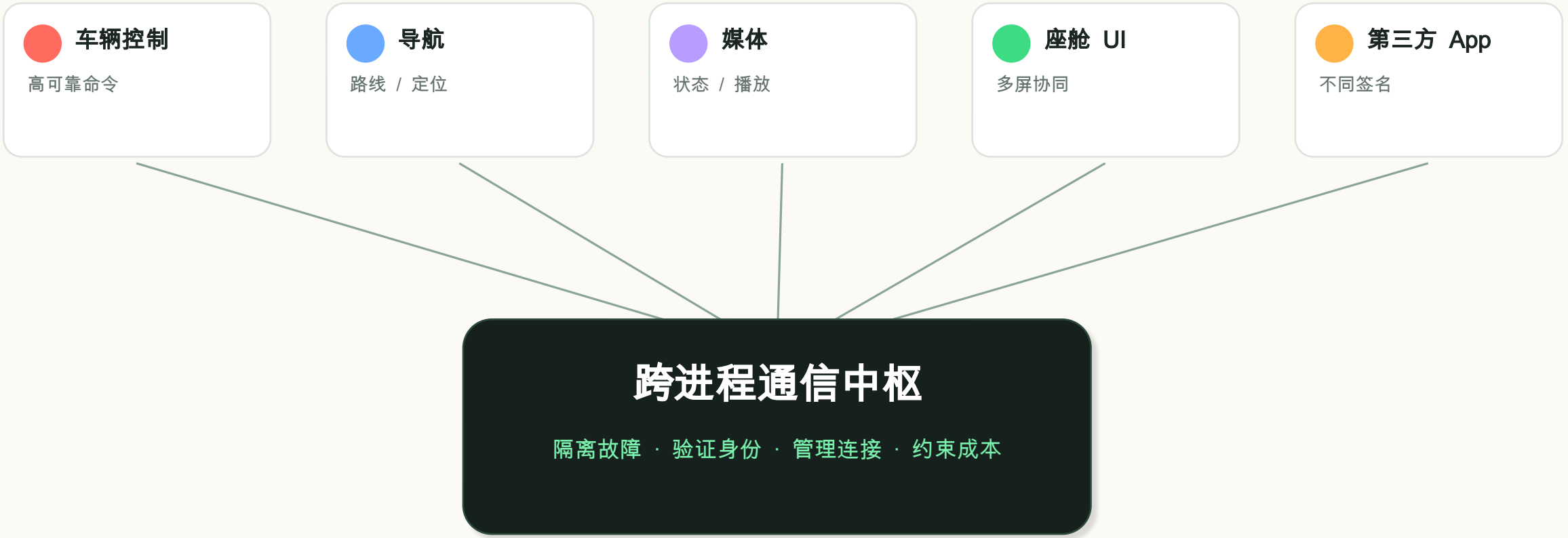
2 控制成本

3 工程治理




45 MIN · TECH SHARING

# 车载系统：多进程不是例外，而是系统形态




**隔离**

一个进程崩溃不拖垮全车

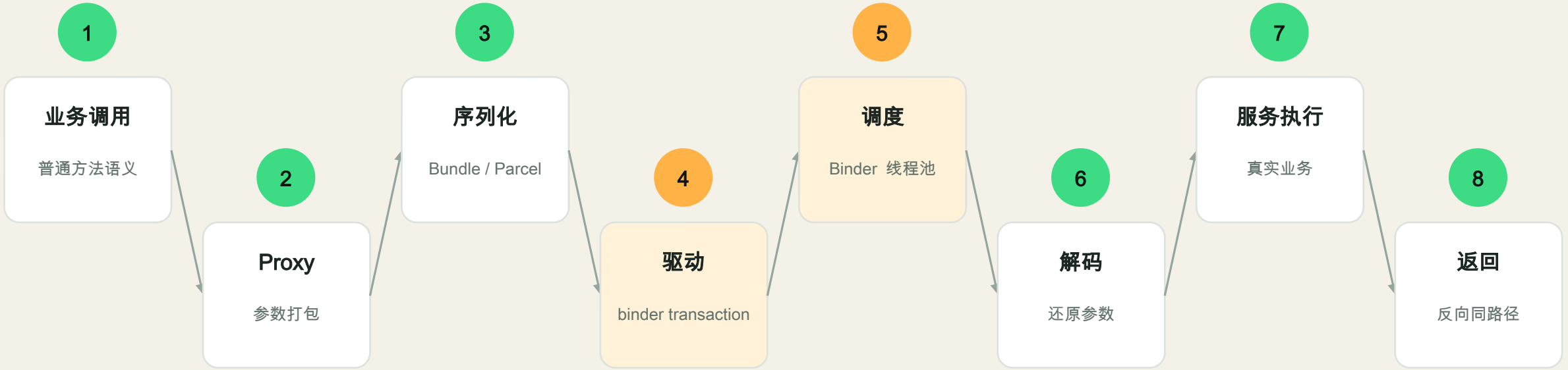
**安全**

不同供应商需要显式互信

**演进**

模块独立交付与升级

# 一次 IPC 调用，至少穿过 8 个成本点




**关键心智： IPC ≠ 本地函数调用**

真正的优化，通常从减少调用次数与数据量开始

# Android IPC ：先按语义分类，再谈性能

RPC

## Binder / AIDL

同步请求响应

消息

## Messenger

串行命令队列

数据访问

## ContentProvider

URI + CRUD

事件通知

## Broadcast

一对多分发

流式 / 跨平台

## Socket

自定义协议

大数据共享

## SharedMemory

共享页 + 同步

**不存在“最快的万能 IPC”**

只有与通信语义、数据规模和拓扑匹配的选择

# Binder ： Android IPC 的“系统总线”




**系统服务几乎都在这条总线上**

# 延迟不是一个数字，而是一组可控成本

## 一次往返的总成本




### 调用次数

通常比单次省 20μs 更重要

### payload

越大，复制与编解码越突出

### 线程模型

串行 Handler 会把并发变成排队

### 尾延迟

平均值掩盖偶发卡顿

# AIDL ：性能基准线，也是工程基线

```
interface IVehicleService {
  VehicleState getState();
  oneway void setMode(int mode);
}
```




平台生成 · 稳定事务码 · 类型安全

## 为什么快

路径短、平台原生、编译期 Stub/Proxy ，额外抽象少。

## 为什么仍然难

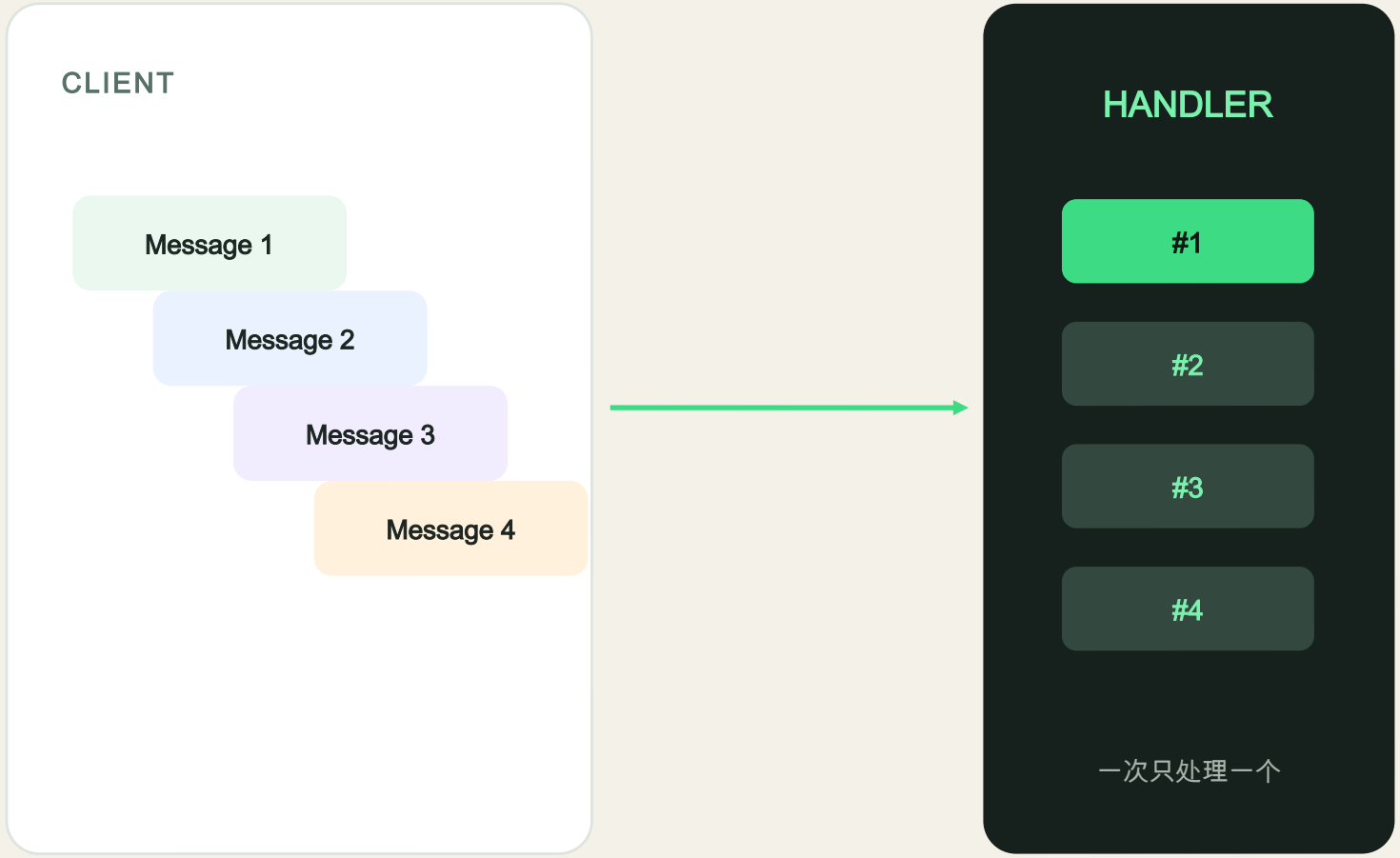
发现、连接、死亡监听、事件、安全策略仍要自行工程化。

## 适用场景

稳定的一对一 RPC ；接口数量少或已有成熟平台层。

基准线 ≠ 万能方案

# Messenger ：简单的代价，是串行队列




**优势**

Message + replyTo ，低频命令很好用。

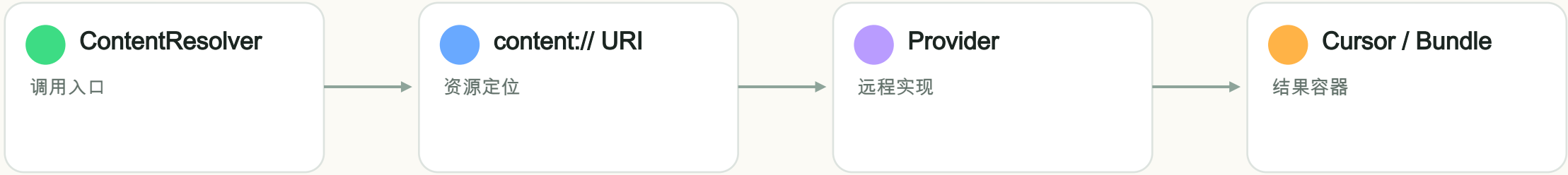
**瓶颈**

单 Handler 排队，高并发会转化为尾延迟。

**适合**

状态机、控制命令、天然需要串行化的业务。

# ContentProvider ：把 IPC 包装成“数据访问”




```
query / insert / update / delete / call
```

**系统集成强**

权限、 URI 、观察者模型成熟

**抽象有成本**

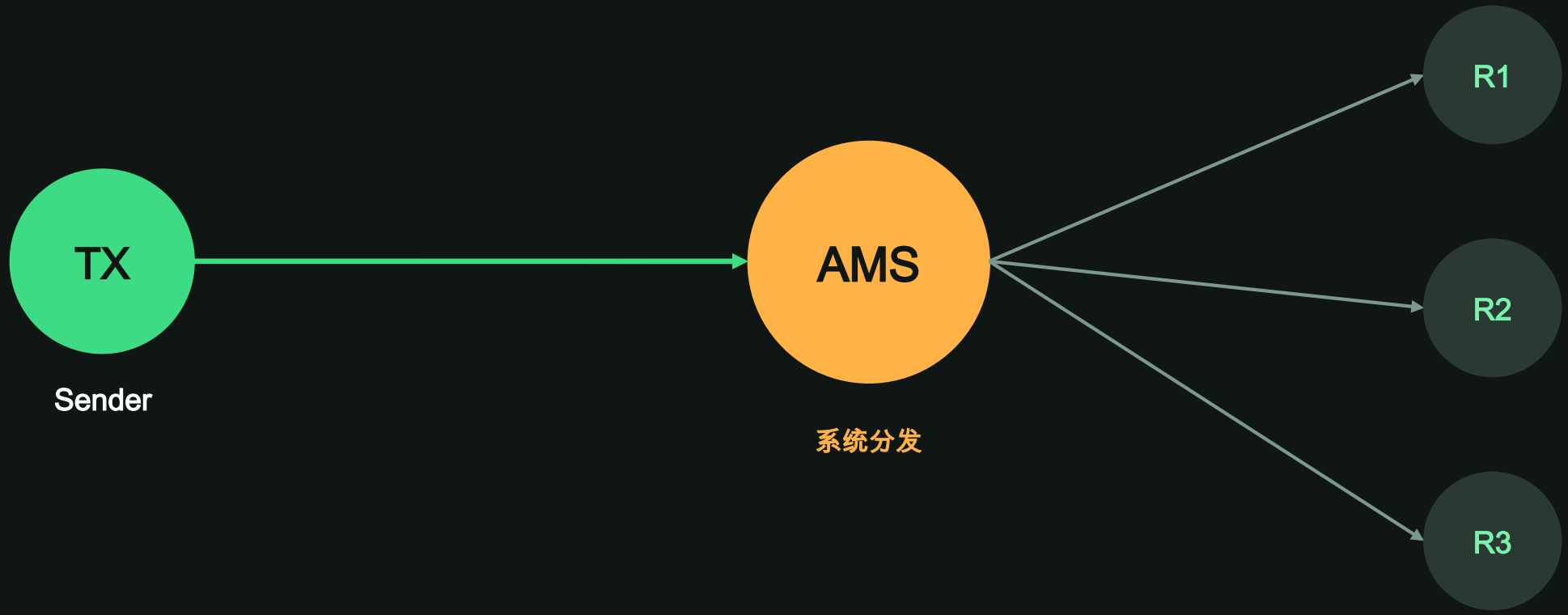
Cursor 、字段转换、两步 echo

**语义要匹配**

数据集合很好，细粒度 RPC 很拧巴

**数据访问能力 > 单次调用极致性能**

# Broadcast ：擅长“告诉大家”，不擅长“问一个答案”




**强项**

一对多、松耦合事件通知

**成本**

AMS 路径、调度波动、后台限制

**误区**

双广播“往返”不等价于 RPC

# Socket ：自由度最高，也意味着责任最多

**业务消息** 你定义语义

**协议帧** 长度 / 版本 / 校验

**连接** 心跳 / 重连 / 超时

**Socket** UDS / TCP

## 何时值得

跨平台、持续流式传输、非 Android 对端、自定义网络协议。

**粘包 / 拆包**

**鉴权 / 加密**

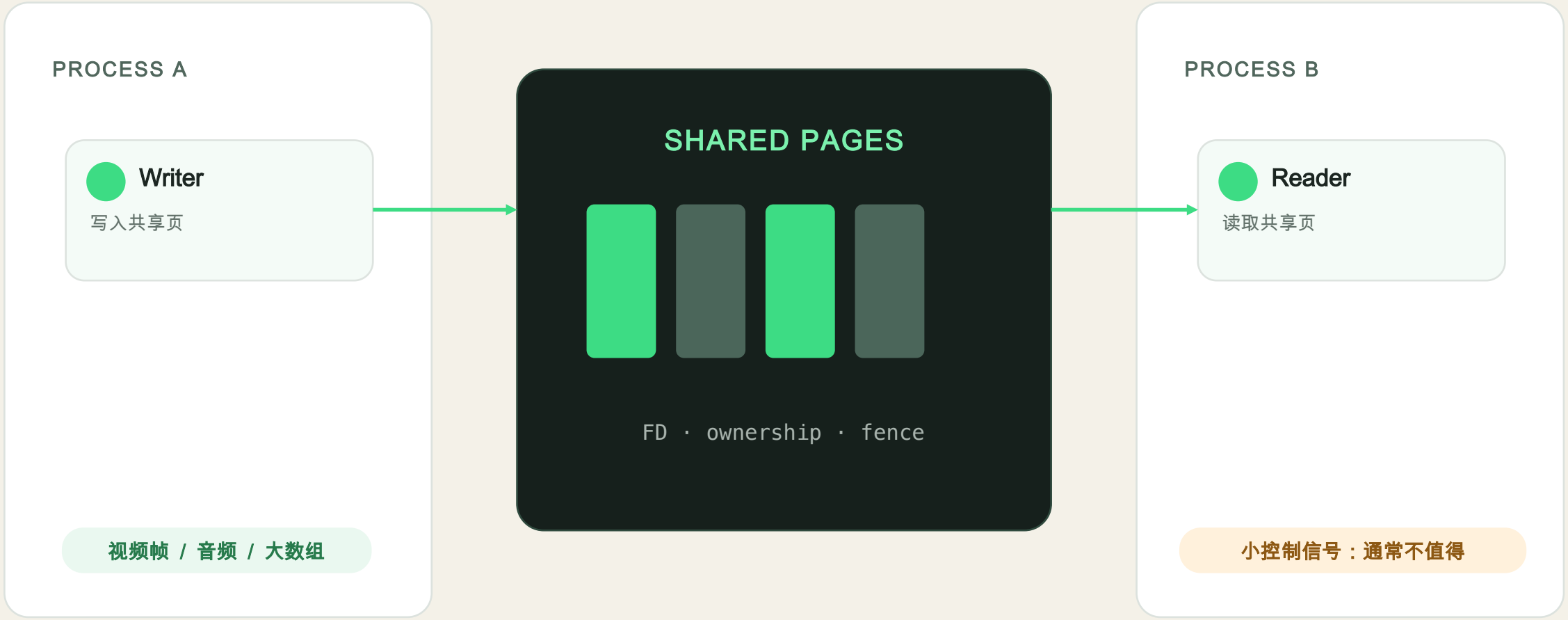
**心跳 / 重连**

**线程 / 背压**

**协议演进**

**自由意味着：平台替你做的事，都要自己实现**

# SharedMemory ：省复制，换来同步与生命周期



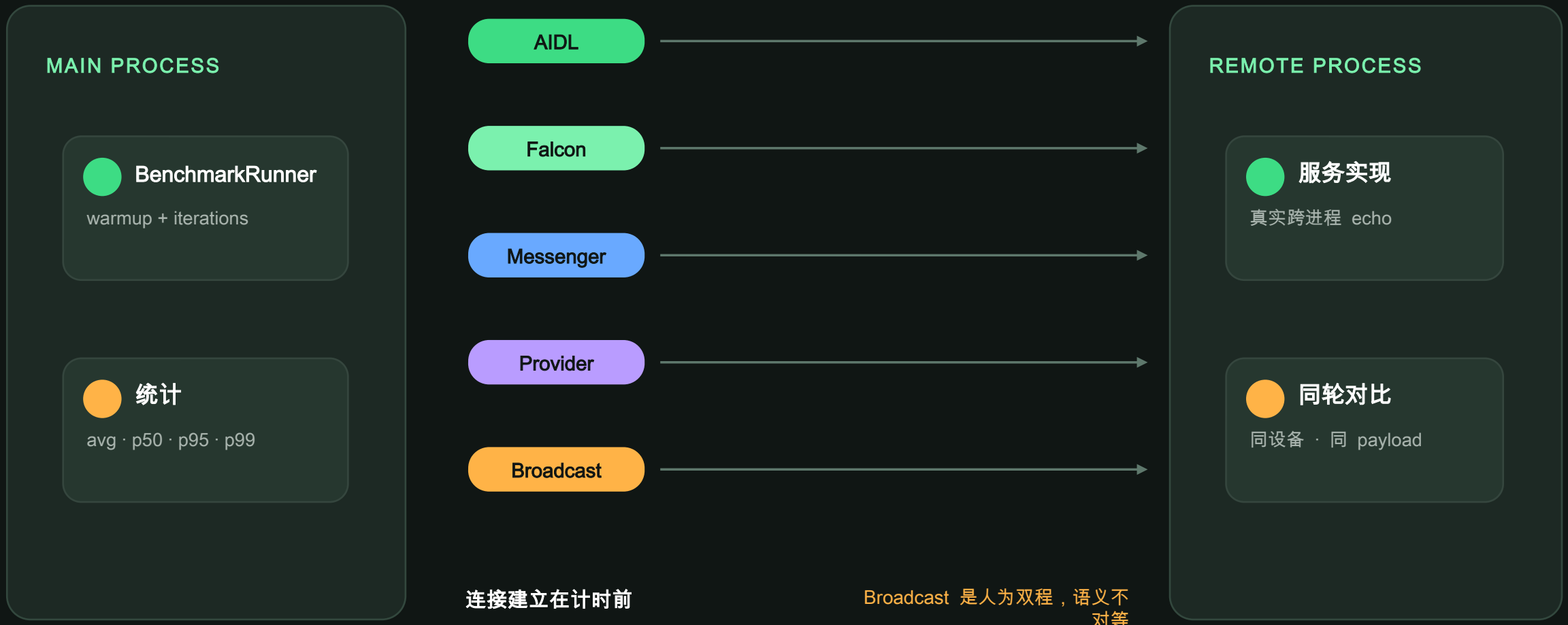

Falcon 已移除 SharedMemory 混合传输：真实场景没有高频大 payload

# 六种原生方式 + Falcon ：一张表做初筛

| 方案 | 核心语义 | 延迟倾向 | 大数据 | 一对多 | 工程成本 | 最佳场景 |
|---|---|---|---|---|---|---|
| AIDL | RPC | 低 | 有限 | 否 | 中 | 稳定接口 |
| Messenger | 消息 | 中 | 不宜 | 否 | 低 | 串行命令 |
| Provider | 数据 | 中高 | 有限 | 否 | 中 | CRUD / 数据共享 |
| Broadcast | 事件 | 高波动 | 不宜 | 是 | 低 | 系统通知 |
| Socket | 流 / 协议 | 可控 | 支持 | 可扩展 | 高 | 跨平台 / 流 |
| SharedMemory | 共享页 | 低 | 强 | 否 | 很高 | 高频大块数据 |
| **Falcon** | **治理型 RPC** | **接近 Binder** | **有限** | **事件流** | **低→中** | **多模块 / 多 App** |

**选型顺序：语义 → 数据规模 → 治理成本 | Falcon 是 Binder 上的工程化层**

# 公平测试：统一“远程进程 + 预热 + 稳态往返”



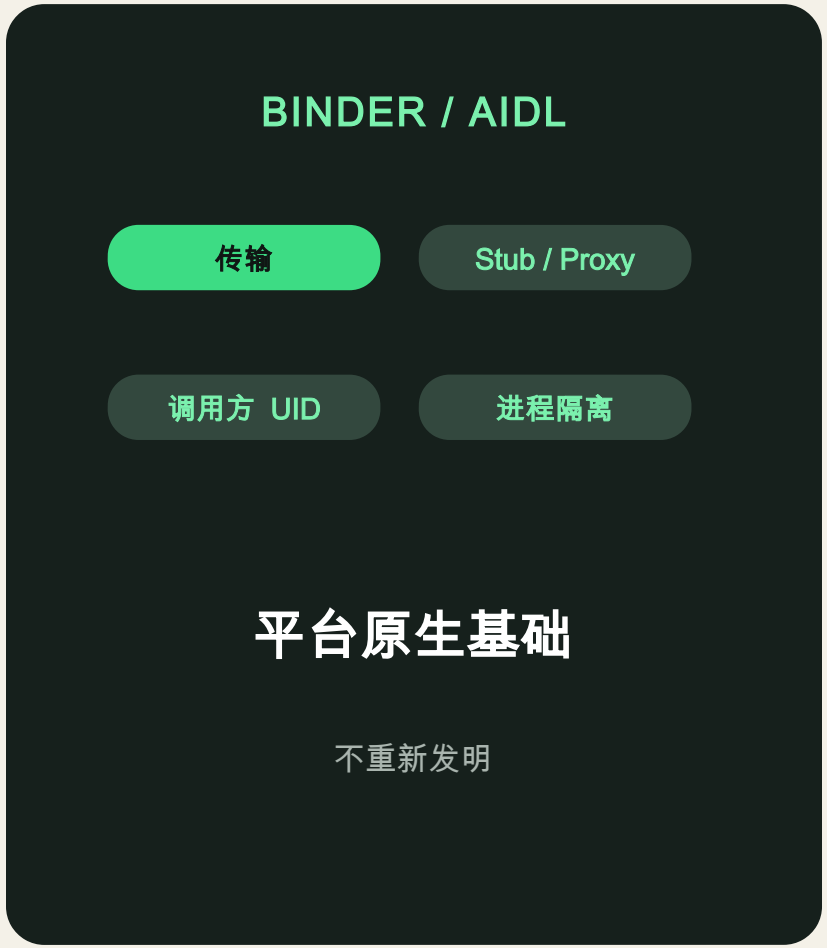

**没有设备实测，就不画“精确柱状图”**

# 优化优先级：先消除工作，再加速工作




**原则：没有测量证据，不升级传输复杂度**

# Binder 解决“能通信”，Falcon 解决“可规模化”




**接口生成**

减少样板

**发现 / 重连**

管理 Binder 生命周期

**事件 / Flow**

统一异步语义

**安全治理**

可信签名与规则

**监控 / 超时**

默认关闭可观测

**多 App 接入**

显式互信

**Falcon = Binder-native 调用链 + 编译期生成 + 运行时治理**

# 业务只看接口；跨进程细节交给生成代码

```
@IpcService
interface VehicleService {
  @IpcMethod suspend fun state(): VehicleState
  @IpcEvent fun events(): Flow<VehicleEvent>
}

Falcon.init(context) {
  generated(AppFalconGeneratedRegistry)
}
falcon.register(VehicleService::class, impl)
```

接口即契约

不写 Stub

Flow 原生

显式 Registry

## 像本地接口一样调用

但在协程 / 后台线程中尊重远程调用语义

# Falcon 架构：编译期与运行时分工明确




**静态生成消除反射分发；运行时只负责连接与治理**

# KSP 生成三件套：Proxy、Dispatcher、Registry

## 1 Xxx_Proxy

- 实现业务接口
- 参数写入 Bundle
- 调用 transport
- 解码结果

CLIENT

## 2 Xxx_Dispatcher

- when(methodId)
- 类型化解码
- 调用 impl
- 编码结果

SERVER

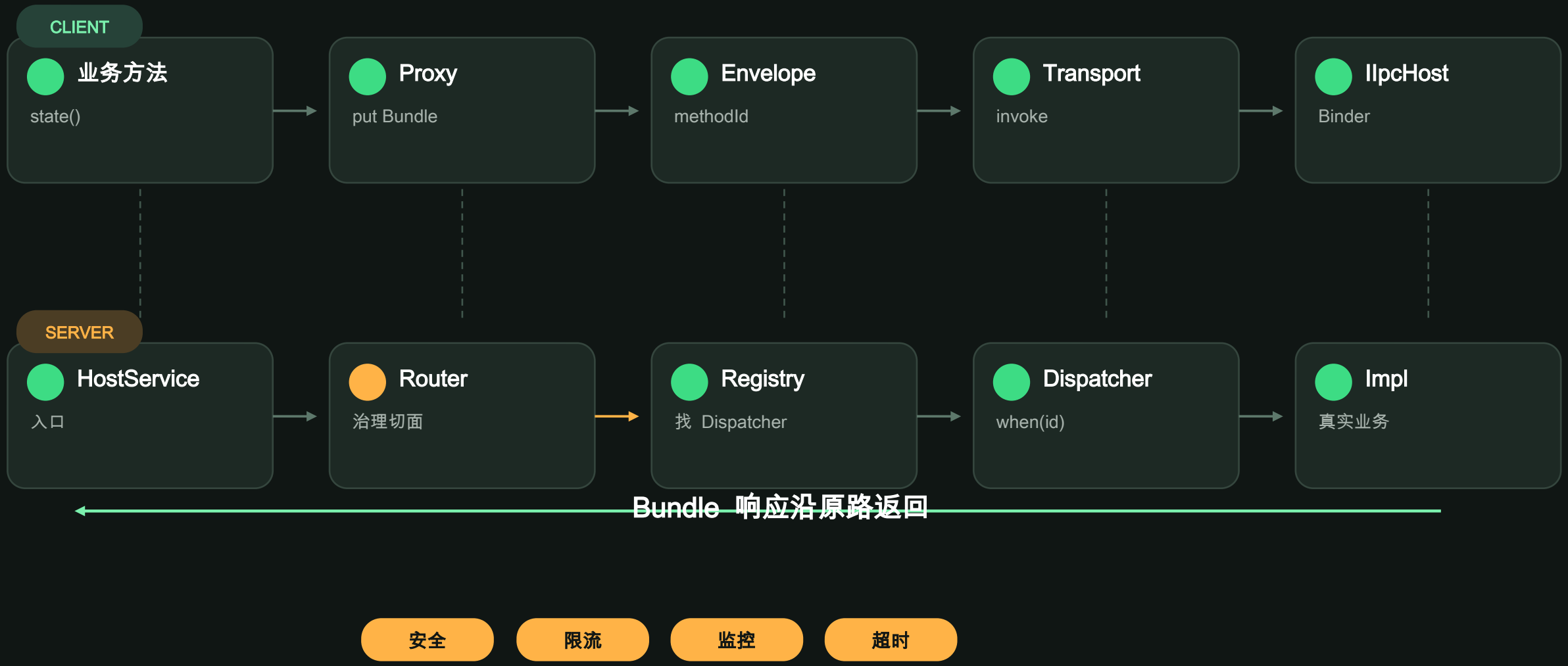
## 3 ModuleRegistry

- proxyFactories
- dispatcherFactories
- schema hashes
- 显式注入

MODULE

不支持的 wire type → 编译期失败

# 一次 Falcon 请求—响应：真正走过的 10 个节点



# 稳定 methodId + schema hash ：匹配“方法”也匹配“数据布局”

**FNV-1a32**

`"state(Int,String)"` → 0x72A9...

重载安全 · 改名即 breaking change

**SCHEMA HASH**

**返回类型 + Parcelable 字段布局**

发现阶段比对一次，避免静默错读

**IpcReply 参数**

不参与 methodId

**哈希碰撞**

KSP 编译失败

**返回类型变化**

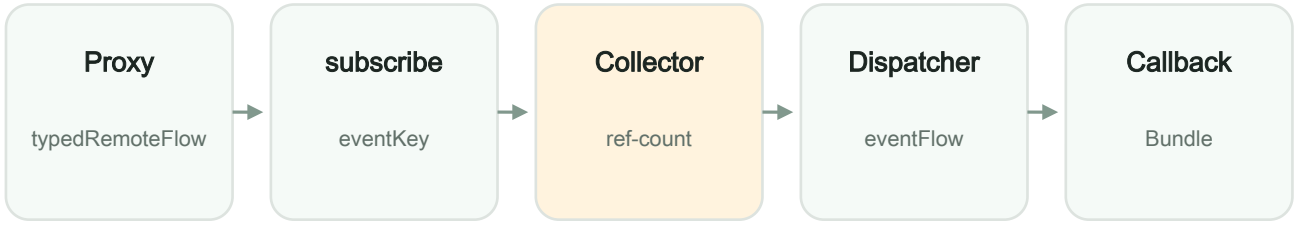
schema hash 捕获

**Bundle 编码**

基础类型 / Parcelable / enum

# Event / Stream / Callback ：统一走生成代码与 Binder 回调

## EVENT / STREAM





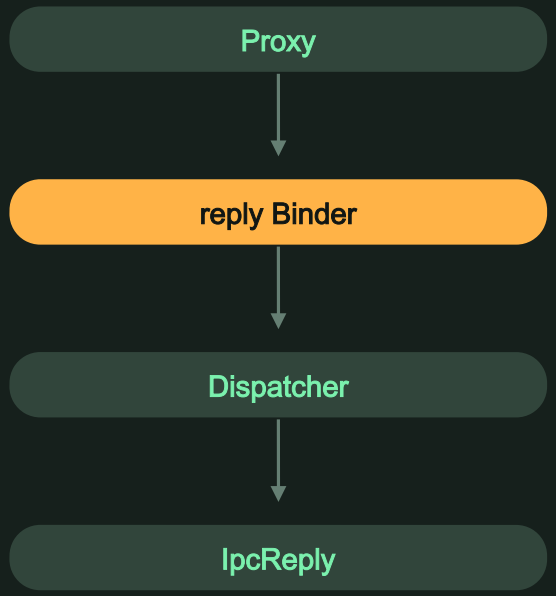


首订阅启动采集

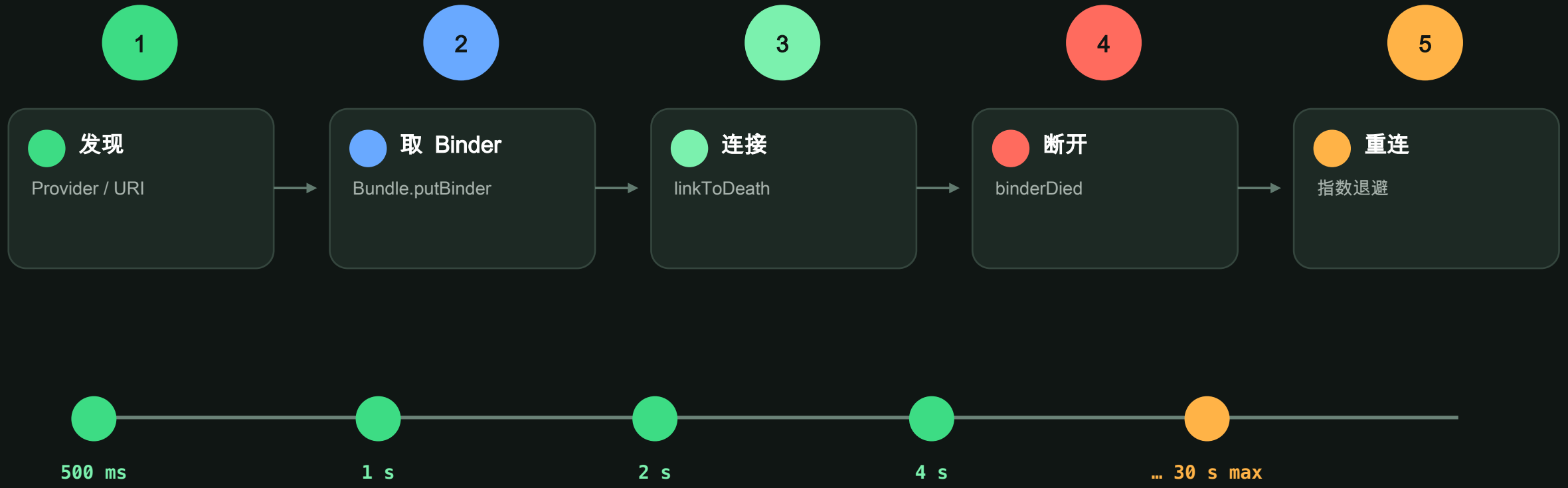
末订阅取消 Job

多个订阅者共享同一服务端 Flow

## CALLBACK



# 发现与重连：连接不是一次动作，而是一个状态机




**每个 peer 独立退避 · 成功后复位 · 死亡回调防重复**

发现成本是生命周期成本，不计入稳态单次 RPC

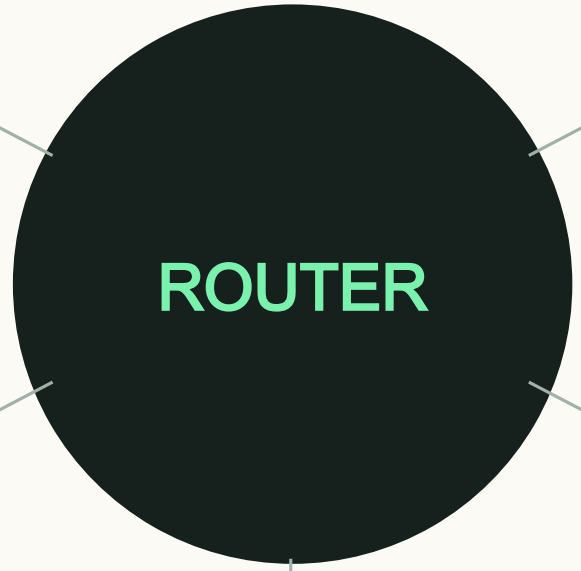
# 安全、稳定、可观测：都在调用入口形成闭环

**签名信任**

自身证书 + trustedSignatures

**限流**

每 PID 滑动窗口




**访问规则**

调用方进程 / 包名

**超时**

调用边界与错误码

**监控**

NONE / BASIC / DETAILED / FULL

默认 MonitorLevel.NONE

# Falcon 的价值与边界：知道什么时候不用它

## 适合

- ✓ **多模块、多进程、多 App**
- ✓ 小到中等 payload 的 Binder RPC
- ✓ 需要事件 / Flow / Callback
- ✓ 需要互信、限流、重连和监控

## 不适合

- × **视频帧 / 高频大数据**
- × 非 Android 跨平台协议
- × 单个简单 AIDL 已足够
- × 试图突破 Binder 事务上限

**Falcon 优化的是工程复杂度，而不是 Binder 的物理上限**

THREE TAKEAWAYS

# 把 IPC 做对，比把 IPC 做快更重要

Q & A

01 **先选语义** RPC 、消息、数据、事件、流与共享内存不是同一道题

02 **再控成本** 减少调用与 payload ，关注排队和 P99

03 **最后治理** 用生成代码、连接管理与安全能力降低系统复杂度

```
Falcon = Binder-native + KSP + Governance
```

谢谢